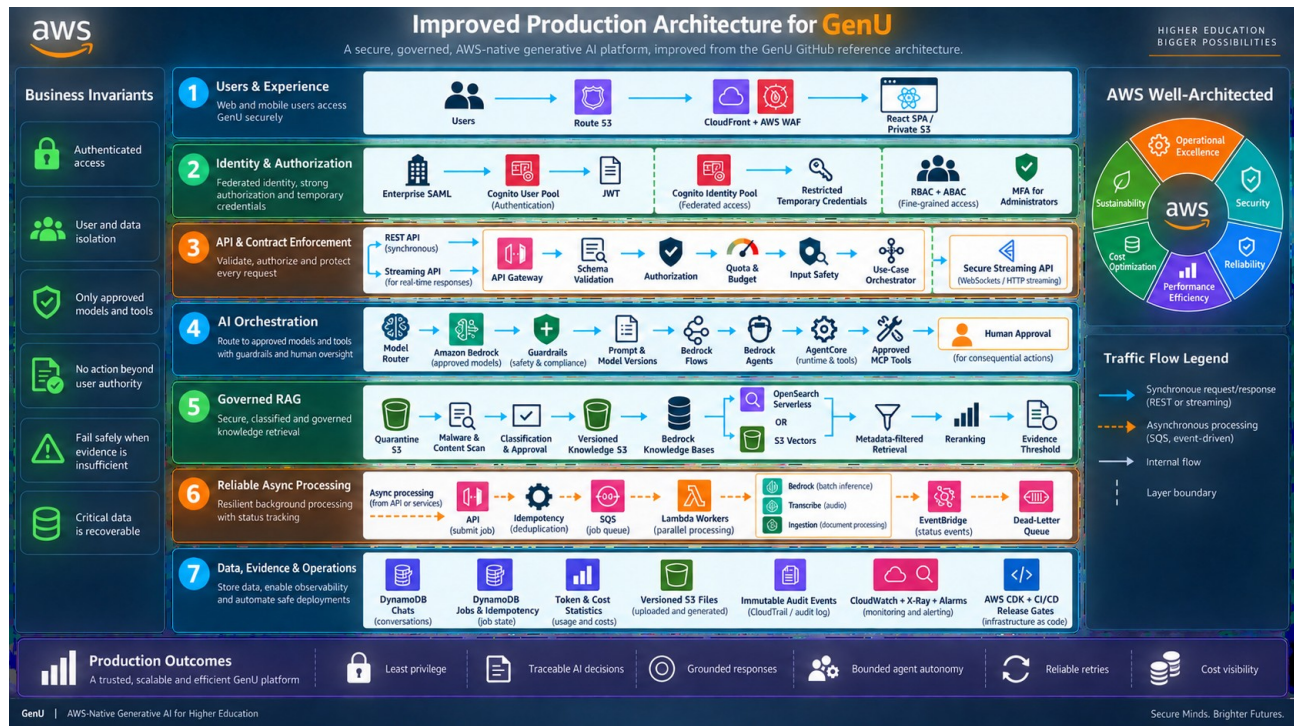


A Design First Approach to GenU

Engineering a governed Amazon Bedrock application from business intent to production evidence



GenU production architecture derived from the current AWS sample and strengthened through explicit contracts and governance.

Case study based on aws-samples generative-ai-use-cases

Prepared for cloud architecture security and generative AI education

September 2026

Executive summary

GenU is an AWS open source application that combines a React interface, Amazon Cognito, API Gateway, AWS Lambda, Amazon Bedrock, retrieval augmented generation, agents, multimodal features and infrastructure as code. Its breadth makes it a useful case study for design-first engineering because a feature-led implementation alone does not establish which outcomes matter, which rules must always hold, or what evidence is required before release.

The design-first approach begins with the business problem and turns it into traceable requirements, invariants and executable contracts before selecting or modifying implementation components. For GenU, this produces clear boundaries around identity, knowledge access, model selection, agent authority, cost, recovery and evidence. The current source provides a strong reference foundation, but production use requires narrower IAM policies, durable data settings, governed document ingestion, asynchronous handling of long-running work, explicit evaluation gates and tested recovery procedures.

The central conclusion is that GenU should be treated as an extensible AI experience platform. Deterministic services must retain control of authentication, authorization, validation, budgets, tool permissions and consequential actions. Model output remains evidence-informed advice or generated content; it does not become an authority boundary.

Study scope and method

This paper examines the GenU repository architecture and applies the AWS Well-Architected Framework and the AWS Well-Architected Generative AI Lens. It separates source-confirmed implementation features from recommended production controls. The analysis follows six design stages: business framing, invariants and roles, contract design, threats and failures, continuous verification, and production evidence.

Area	Source confirmed baseline	Design first extension
Experience	React SPA with chat, RAG, media, voice and agent use cases	Use-case outcomes, user roles and permitted capabilities
Identity	Cognito User Pool, Identity Pool and optional SAML	RBAC, ABAC, MFA and resource ownership checks
Application	API Gateway, Express Lambda and direct streaming Lambda	Contract enforcement, bounded time and idempotency
AI	Bedrock models, Guardrails, Flows, Agents and AgentCore	Model allowlists, output schemas, approval gates and evaluations
Knowledge	Knowledge Bases, Kendra, OpenSearch Serverless or S3 Vectors	Governed ingestion, provenance, access filtering and evidence thresholds
Operations	CDK, CloudWatch and token statistics	Release gates, recovery tests, budgets

Area	Source confirmed baseline	Design first extension
		and audit evidence

1 GenU as the case study

1.1 Application purpose

GenU addresses the repeated engineering work that occurs when organizations build separate applications for chat, summarization, RAG, document analysis, media generation and agents. It provides a common interface and shared AWS foundation so approved use cases can reuse identity, storage, model invocation and monitoring services.

1.2 Current source architecture

The current repository is a TypeScript monorepo. The web package contains the React application. The CDK package builds the frontend, identity, APIs, data stores and optional AI stacks. The main REST surface is an Express application running in Lambda through the Lambda Web Adapter. The browser uses a second path for streamed responses by obtaining temporary AWS credentials from a Cognito Identity Pool and invoking a streaming Lambda directly.

Layer	Principal components	Purpose
Edge	CloudFront, private S3, optional WAF	Deliver the SPA and apply edge controls
Identity	Cognito User Pool, Identity Pool, SAML	Authenticate users and issue tokens or temporary credentials
API	API Gateway, Express Lambda, streaming Lambda	Serve CRUD APIs and streamed model output
AI	Bedrock models, Guardrails, Flows, Agents, AgentCore	Generate responses and execute approved AI workflows
RAG	Knowledge Bases, Kendra, OpenSearch Serverless, S3 Vectors	Retrieve organizational knowledge
Data	DynamoDB and multiple private S3 buckets	Persist chats, statistics, files, media and source documents
Operations	CDK, CloudWatch and tags	Deploy, observe and allocate cost

1.3 Architectural implications

GenU has two trust paths. REST requests are protected by the API Gateway Cognito authorizer, while direct streaming and selected real-time functions use temporary IAM credentials. A secure review must therefore cover both JWT authorization and the authenticated Identity Pool role. The optional nature of RAG, agents, MCP, media and closed-network stacks also means there is no single GenU risk profile; the deployed configuration defines the actual attack surface and cost model.

2 Business problem and desired outcomes

2.1 Problem statement

Organizations need a controlled way to experiment with and deploy generative AI. Separate applications create duplicated infrastructure, uneven security, inconsistent prompt handling, fragmented cost reporting and unclear ownership. A shared platform can reduce this duplication, but it also concentrates access to models, documents and tools. The design must therefore optimize for governed reuse rather than unrestricted feature availability.

2.2 Stakeholders

Stakeholder	Need	Decision or responsibility
End user	Useful and timely AI assistance	Select an approved use case and verify important output
Use-case designer	Reusable prompts and interfaces	Define purpose, input, output and evaluation contract
Knowledge owner	Controlled use of organizational documents	Approve sources, metadata, access scope and retention
AI engineer	Reliable model and retrieval integration	Implement adapters, evaluation and bounded fallbacks
Security administrator	Limited access and auditable actions	Approve roles, tools, networks and logging policy
Operations engineer	Observable and recoverable service	Own alarms, runbooks, quotas and recovery exercises
Governance reviewer	Traceable AI behavior	Review evidence before release and after material changes

2.3 Business requirements

ID	Requirement	Acceptance evidence
BR 01	Provide one authenticated interface for approved generative AI use cases.	Only enabled use cases appear and protected operations reject unauthenticated access.
BR 02	Support approved models without binding the platform to one model version.	Deployment rejects unsupported models and records model identity per request.
BR 03	Ground domain answers in authorized organizational knowledge.	Citations map to retrieved passages and access isolation tests pass.
BR 04	Allow authorized designers to create versioned reusable use cases.	Ownership, sharing scope, version and approval state are stored.
BR 05	Permit agents to use tools only within the user and use-case authority.	Denied actions never invoke tools and every attempt is audited.
BR 06	Protect prompts, files, conversations and knowledge sources.	Encryption, masking, retention and deletion tests pass.
BR 07	Attribute and control model consumption.	Usage is reported by user, model and use case; budget alarms are tested.
BR 08	Recover critical configuration and data.	Restore exercises meet documented recovery objectives.

3 Roles assumptions and invariants

3.1 Role model

Role	Permitted responsibilities	Explicit exclusions
User	Run approved use cases and manage owned conversations	Cannot publish use cases or access another user's private data
Designer	Draft and test use-case definitions	Cannot approve own production release
Knowledge owner	Approve sources and metadata policies	Cannot bypass access filters
Administrator	Configure platform features and identity integration	Cannot use model output to grant permissions
Tool approver	Approve agent tools and	Cannot broaden user authority

Role	Permitted responsibilities	Explicit exclusions
	consequential actions	through an agent
Auditor	Read evidence and control results	Cannot alter evidence records

3.2 Business invariants

Invariants express rules that must remain true across model changes, retries, interfaces and deployment modes. They are stronger than prompt instructions because deterministic controls enforce them.

ID	Invariant	Primary enforcement
INV 01	Every protected operation is linked to an authenticated identity.	Cognito, API Gateway and IAM
INV 02	A user cannot access another user's private conversations, files or prompts without an explicit share.	Backend ownership checks
INV 03	Retrieval never returns content outside the user's authorized scope.	Identity-derived metadata filters
INV 04	Only approved models, prompts, tools and knowledge versions are used.	Configuration registry and deployment gates
INV 05	Model output is untrusted until validated for its intended use.	Output schema and policy validation
INV 06	A model or agent cannot create authority beyond the authenticated user.	IAM and policy engine
INV 07	Consequential actions require explicit human approval.	Approval state machine
INV 08	A grounded claim cites evidence actually returned by retrieval.	Passage identifiers and evidence validation
INV 09	A retry cannot create a duplicate business outcome.	Idempotency record and conditional write
INV 10	Critical production data survives ordinary stack replacement or deletion.	Retention, backup and restore controls
INV 11	Every governed response can be traced to its model, prompt, guardrail	Evidence record

ID	Invariant	Primary enforcement
	and data configuration.	
INV 12	If authorization, grounding or a required safety control is unavailable, the operation fails safely.	Fail-closed policy

4 Contract first design

4.1 Contract hierarchy

The business contract defines the outcome and authority boundary. Each use-case contract narrows permitted input, output and evidence. API, retrieval, model and tool contracts make those obligations testable. Operational contracts then define service levels and recovery behavior.

1. Business contract defines purpose, actors, protected data and approval responsibility.
2. Use-case contract defines input, output, preconditions, failure states and evidence.
3. API contract defines schemas, authentication, status codes and idempotency.
4. Retrieval contract defines authorized sources, filtering, relevance and citations.
5. AI output contract defines structure, prohibited decisions and review requirements.
6. Tool contract defines permissions, limits, approval and audit fields.
7. Operational contract defines latency, availability, recovery and cost thresholds.

4.2 Grounded chat use-case contract

Contract element	Definition
Purpose	Answer an authenticated user's question using authorized organizational knowledge.
Input	Question from 1 to 4,000 characters, selected approved model and use-case version.
Preconditions	Identity valid; use case enabled; model approved; knowledge service available; access scope resolved.
Success	Answer, citations, request identifier, model identifier, prompt version and usage metadata.
Failure states	Unauthorized, insufficient evidence, safety blocked, throttled, validation failed or service unavailable.
Postcondition	The response and evidence status are recorded without exposing prohibited content.

Contract element	Definition
Human responsibility	The user verifies material claims before consequential use.

4.3 AI output contract

A structured response should contain the answer, citations, limitations, model identity, prompt version and human-review flag. Citations must refer to passage identifiers returned by the retrieval service. The model cannot set authorization decisions, approval state or official audit fields. Invalid output receives a validation-failed status rather than silent repair when the field affects a consequential decision.

4.4 Agent tool contract

Field	Required definition
Identity	Calling user, agent and session
Authorization	Required scope and deterministic enforcing service
Input	Typed schema, length limits and prohibited values
Execution limits	Timeout, maximum calls, retries, token and cost budget
Approval	Risk class and human approval requirement
Output	Typed result and verification rule
Audit	Actor, tool version, parameter hash, decision, result and duration

5 Improved production architecture

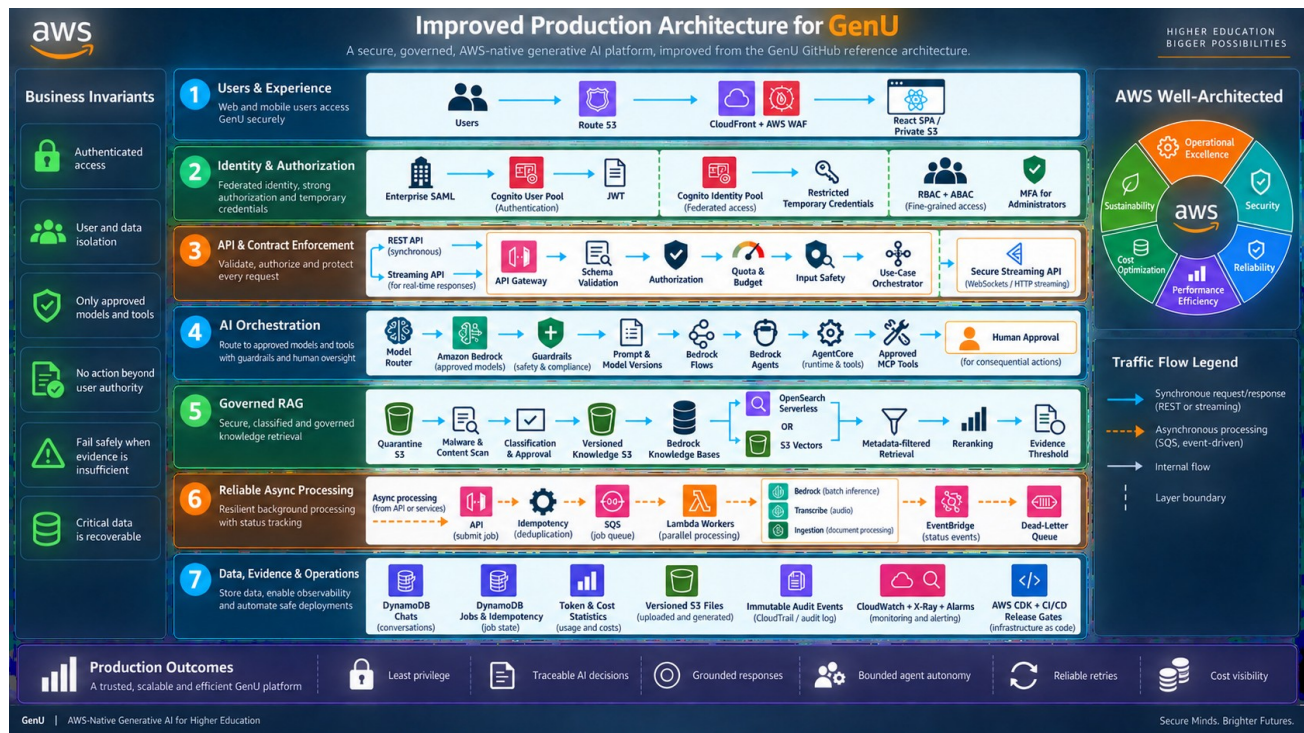


Figure 1 Improved production architecture for GenU

5.1 Contract enforcement

Each request passes through schema validation, authorization, quota checks and input safety controls before model, retrieval or tool execution. The output then passes through schema and policy validation before it is streamed or stored. This arrangement prevents natural-language instructions from becoming authorization rules.

5.2 Governed knowledge ingestion

Documents first enter a quarantine bucket. Automated checks validate format, malware status and prohibited content. A knowledge owner assigns classification, access metadata, validity period and version before approval. Only approved content enters the Knowledge Base data source. Retrieval filters are derived from identity claims rather than supplied by the user.

5.3 Reliable asynchronous work

Video generation, transcription, large-document ingestion and long agent research should use an asynchronous job contract. The API creates an idempotency record and queues the work. Workers report explicit states through EventBridge. Exhausted retries move the job to a dead-letter queue. The user sees accepted, queued, processing, completed, failed or cancelled rather than an ambiguous timeout.

6 Threat and failure design

6.1 Threat model

Threat	Control boundary	Verification evidence
Prompt injection in user input	Input isolation, tool policy and no implicit trust in retrieved instructions	Adversarial prompt suite
Knowledge poisoning	Quarantine, source approval, provenance and revocation	Ingestion audit and malicious-document tests
Cross-user disclosure	Ownership checks and identity-derived metadata filters	Negative authorization tests
Excessive agent authority	Scoped role, tool allowlist, bounded plan and human approval	Denied-action and privilege-escalation tests
Sensitive prompt logging	Field classification, masking and retention policy	Log inspection tests
Cost exhaustion	Token, turn, tool and per-user budgets	Quota and anomaly-alarm tests
External MCP compromise	Allowlist, version pinning, controlled egress and circuit breaker	Dependency and failure-injection tests
Destructive deployment	Retention policy, backups and protected production pipeline	Deletion-protection and restore tests

6.2 Failure behavior

Failure	Required response
Authentication missing	Return 401 and do not invoke the model.
Authorization denied	Return 403 without revealing restricted resource details.
No relevant evidence	Return insufficient evidence; do not present an authoritative grounded answer.
Model throttled	Use bounded retry and approved fallback or return an explicit retryable failure.
Tool timeout	Stop the tool, retain evidence and return partial or failed state.
Output schema invalid	Reject downstream use and record validation failed.
Duplicate request	Return the existing result or processing status.

Failure	Required response
Safety service unavailable	Fail closed for use cases that require the control.

Plain-language contract: when a contract is violated, GenU stops at the affected boundary, reports the failure without exposing sensitive details, preserves evidence and does not claim that the task succeeded.

7 AWS Well Architected alignment

The AWS Well-Architected Framework defines six pillars. The Generative AI Lens extends them with model quality, prompt security, retrieval performance, agent safety, responsible AI and lifecycle governance. GenU aligns with the framework through managed services and CDK, but production assurance depends on configuration and operational evidence.

Pillar	Existing GenU strength	Required production improvement
Operational Excellence	CDK, configurable stacks, CloudWatch option and usage statistics	CI/CD gates, versioned AI assets, alarms, runbooks and rollback
Security	Cognito, SAML, WAF, private S3, Guardrails and closed-network option	Narrow IAM, restrict CORS, private retrieval, MFA and governed logging
Reliability	Managed serverless services and on-demand DynamoDB	PITR, S3 versioning, retention, queues, DLQs and recovery tests
Performance Efficiency	Streaming, managed inference, reranking and selectable vector stores	Load tests, quota planning, retrieval tuning and task-based model routing
Cost Optimization	Pay-per-use compute, token statistics, tags and Kendra scheduling	Budgets, cost per outcome, token limits and idle-resource review
Sustainability	Serverless scaling and choice of model size	Smaller approved models, bounded agents, deduplicated ingestion and lifecycle policies

7.1 Source-derived hardening priorities

Priority	Change	Reason
1	Replace broad Bedrock and log permissions with resource-scoped	Reduce compromise blast radius.

Priority	Change	Reason
	actions.	
2	Enable DynamoDB PITR and retain critical production buckets and tables.	Prevent avoidable data loss.
3	Restrict CORS and direct browser IAM permissions.	Protect the second trust path.
4	Add evaluation and security gates before deployment.	Prevent unmeasured prompt, model or retrieval regressions.
5	Move long-running work to queues with idempotency and DLQs.	Make retries and failure states reliable.
6	Govern knowledge ingestion and retrieval authorization.	Protect confidentiality and answer provenance.

8 Continuous verification

8.1 Verification strategy

Test layer	Examples	Release evidence
Contract	Schema, status codes, idempotency and ownership	Automated test report
Security	Prompt injection, cross-user access, tool escalation and log masking	Security test results and exceptions
RAG	Retrieval precision, citation support and metadata isolation	Golden-query evaluation
Model quality	Correctness, groundedness, harmful output and refusal quality	Versioned evaluation dataset and score
Reliability	Throttling, timeout, dependency failure, duplicate delivery and restore	Failure-injection and recovery report
Performance	Time to first token, API latency, concurrency and retrieval latency	Load-test report
Cost	Tokens, tool calls and cost per successful outcome	Budget comparison and approval

8.2 Suggested measurable contracts

- All protected API operations reject missing or invalid identity.
- Authorization tests produce zero successful cross-user or cross-scope reads.
- Non-generation REST endpoints maintain P95 server-side latency of no more than 1,000 milliseconds under the defined load, with an error rate below 1 percent.
- Every governed model response records the request, user, use case, model, prompt and guardrail versions.
- Every grounded material claim links to an actually retrieved passage.
- Every consequential agent action has an approval record before execution.
- Critical data restoration meets the documented recovery time and recovery point objectives.
- Every model invocation is attributable to a user, model and use case.

8.3 Release gate

A release may proceed only when API contracts, infrastructure checks, least-privilege review, adversarial security tests, RAG authorization tests, quality evaluation, performance limits, estimated cost and rollback readiness meet their approved thresholds. The gate records the exact code, prompt, model, guardrail, dataset and knowledge versions evaluated.

9 Production evidence and operations

9.1 Evidence record

A governed interaction should record a request identifier, authenticated actor, use-case version, model identifier, prompt version, guardrail version, knowledge-base identifier, retrieved passage identifiers, tool calls, token usage, latency, validation status and completion status. Sensitive prompt content should be excluded or masked according to policy.

9.2 Operational dashboards

Dashboard	Minimum measures	Decision supported
Service health	Request rate, error rate, latency, throttling and queue depth	Incident response and capacity
Model quality	Evaluation score, refusal rate, citation support and user feedback	Model or prompt release
Retrieval	No-result rate, relevance, reranking latency and access denials	Knowledge and retrieval tuning
Agent safety	Tool calls, denials, approvals, loop limits and failures	Tool governance
Cost	Tokens, media jobs, vector capacity and cost by use case	Budget and model routing

Dashboard	Minimum measures	Decision supported
Data governance	Ingestion status, document versions, retention and deletion	Knowledge owner assurance

9.3 Operating model

Use-case designers own purpose, expected behavior and evaluation examples. Knowledge owners approve sources and access metadata. AI engineers own model and retrieval adapters. Security approves roles, tools and data handling. Operations owns alarms, runbooks, quotas and recovery. Governance reviewers approve production evidence and material changes. Separating these responsibilities prevents the person who creates a use case from being the sole approver of its risks.

10 Implementation roadmap

Phase	Scope	Exit criteria
1 Baseline	Deploy minimal chat with one approved model, Cognito and monitoring.	Authentication, usage and error evidence verified.
2 Contracts	Add schema, ownership, quotas, version registry and evidence record.	Contract and negative authorization tests pass.
3 Governed RAG	Add quarantine, approval, metadata filtering and citation validation.	Golden-query and access-isolation thresholds pass.
4 Bounded agents	Enable selected read-only tools, limits and audit.	Unauthorized and loop-limit tests pass.
5 Consequential actions	Add approval workflow and scoped write tools.	No action executes without required approval.
6 Production resilience	Enable retention, PITR, queues, DLQs, alarms and restore exercises.	Recovery and failure-injection targets pass.
7 Continuous improvement	Review quality, security, cost and sustainability evidence.	Changes remain traceable and within approved thresholds.

10.1 Recommended initial boundary

The first production pilot should enable authenticated chat, one approved text model, one governed Knowledge Base, CloudWatch metrics and a tailored Guardrail. Agent write tools, public MCP servers, code execution, image generation and video generation should remain disabled until their contracts, permissions, cost limits and failure tests are approved.

11 Conclusion

GenU provides a capable AWS reference implementation for a shared generative AI platform. A design-first approach makes the platform governable by defining the business outcome, authority boundaries and evidence before expanding features. The resulting architecture preserves GenU's flexibility while placing deterministic controls around identity, retrieval, model use, agent actions, cost and recovery.

The most important architectural decision is to keep authorization and consequential actions outside the model. Models may interpret, propose and generate. Application services decide what data may be retrieved, which tools may run, whether approval is required and what evidence must be retained. This separation allows GenU to support experimentation without weakening operational accountability.

References

1. Amazon Web Services. Generative AI Use Cases GenU. GitHub repository.
<https://github.com/aws-samples/generative-ai-use-cases>
2. Amazon Web Services. Generative AI Use Cases GenU documentation. <https://aws-samples.github.io/generative-ai-use-cases/en/>
3. Amazon Web Services. AWS Well-Architected Framework.
<https://docs.aws.amazon.com/wellarchitected/latest/framework/the-pillars-of-the-framework.html>
4. Amazon Web Services. AWS Well-Architected Generative AI Lens. Publication date 19 November 2025. <https://docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens/generative-ai-lens.html>
5. OWASP Foundation. OWASP Top 10 for Large Language Model Applications.
<https://owasp.org/www-project-top-10-for-large-language-model-applications/>